

1. Title, goal statement and conventions

Project title: Ad-Vocates

Team Members: Dylan Shackleford, Max Marsh

Goal: Create a reliable and efficient tool that improves browsing performance while giving users greater privacy, security, and control over their web experience.

Conventions: Solid boxes represent components built by the team, circles represent external dependencies the team does not build, arrows for the D0 diagram represent interfaces, arrows for the data-flow diagram represent data with its form labeled

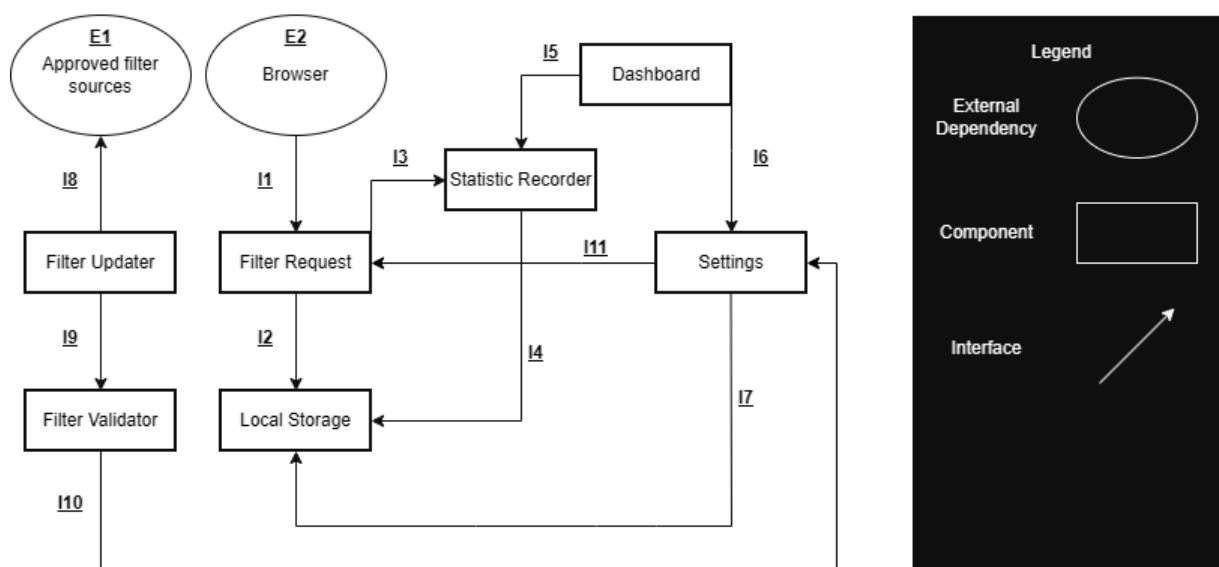
Basic input: Browser request data, user protection settings and filter rules from approved sources

Basic output: Block/allow output, and dashboard showing blocked requests and estimated data saved

2. Block diagram (the D0 diagram)

Ad-Vocates

Goal: Create a reliable and efficient tool that improves browsing performance while giving users greater privacy, security, and control over their web experience.



3. Component responsibility table

Component	Interface in/out	Primary owner	User story
Dashboard - present controls, settings and statistics to user	In : None Out: I5, I6	Max Marsh	US-02, 03, 05
Settings - manage filters and protection settings	In: I6, I10 Out: I7, I11	Max Marsh	US-03, 04, 05
Statistic Recorder - produce diagnostic stats from blocking event data	In: I3, I5 Out: I4	Dylan Shackleford	US-01, 02
Filter Request - block or allow browser request based on filters/settings	In: I1, I11 Out: I2, I3	Max Marsh	US-01, 03, 05
Local Storage - persistent app records stored on user machine	In: I2, I4, I7 Out: None	Max Marsh	US-02, 03, 04, 05
Filter Updater - fetch updated/current filter data from approved sources	In: None Out: I8, I9	Dylan Shackleford	US-04
Filter Validator - validate safety of retrieved filter set	In: I9 Out: I0	Dylan Shackleford	US-04

4. Interface specification table

ID	Named and typed in/out	Data format	Protocol	Error behavior / handler
I1 Browser to Filter Request	In : requestId: string, requestUrl: string, resourceType: string, pageHost:	Structured object	Browser extension request	Filter Request allows request if filtering fails and tries to log error in Local Storage

	string Out: result: enum(allowed, blocked)			
I2 Filter Request to Local Storage	In: action: enum(readFilters , logError), errorMsg: string (opt) Out: filters: string[], status: enum(success, failed)	JSON	Local read/write	Filter Request keeps current filters if read fails, if log fails filtering continues
I3 Filter Request to Statistic Recorder	In: eventId : string, category : enum(ad, tracker, redirect, other), estimatedBytes: int Out: None	JSON	Local internal event	Statistic Recorder ignores duplicate events and continues if recording fails
I4 Statistic Recorder to Local Storage	In: action: enum(read, write), blockedTotal : int (opt), estimatedBytes: int (opt) Out: blockedTotal: int, estimatedBytes: int, status: enum(success, failed)	JSON	Local read/write	Statistic Recorder keeps current total in memory if save fails and reports unavailable statistics if read fails
I5 Dashboard to Statistic Recorder	In: query: enum(getTotal) Out: blockedTotal : int, estimatedBytes: int, status:	JSON	Local query/response	Dashboard shows “Statistics currently unavailable” if query fails

	enum(success, failed)			
I6 Dashboard to Settings	In: action: enum(read,write), enabled: bool (opt), allowlist: string[] (opt) Out: enabled: bool, allowlist: string[], status: enum(success, failed)	JSON	Local request/response	Settings rejects invalid addresses, keeps prior config, and returns error to Dashboard
I7 Settings to Local Storage	In: action: enum(read,write), enabled: bool (opt), allowlist: string[] (opt), filters: string[] (opt) Out: enabled: bool, allowlist: string[], filters: string[], status: enum(success, failed)	JSON	Local read/write	Settings keeps prior config if save fails and reports failure through Dashboard
I8 Filter Updater to Approved Sources	In: sourceUrl: string Out: filterText: string, statusCode: int	Plain text filter	HTTPS request	Filter Updater retries a set amount of hours later if download fails and keeps existing filters
I9 Filter Updater to Filter Validator	In: sourceUrl: string, filterText: string Out: valid: bool, errorMsg: string (opt)	Structured object	Local validation request/response	Filter Validator rejects unapproved sources or invalid filters, existing filters stay active
I10 Filter Validator to Settings	In: filters: string[] Out: status: enum(success, fai	JSON	Local request/response	Settings keeps previous filters if new validated filters cannot be

	led)			saved/applied
I11 Settings to Filter Request	In: enabled: bool, allowlist: string[], filters: string[] Out: status: enum(success,fai led)	JSON	Local request/response	Filter Request keeps previous config if new config fails, Settings reports error to Dashboard

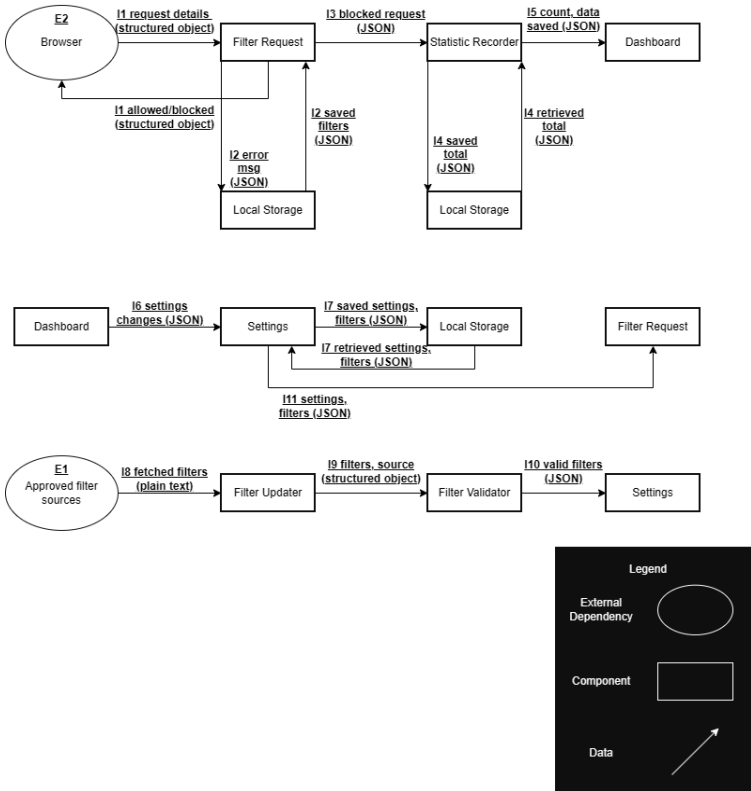
An example payload for I3 would be the following:

```
{
  "eventId": "request1",
  "category": "ad",
  "estimatedBytes": 1234
}
```

5. Data-flow diagram

Ad-Vocates

Goal: Create a reliable and efficient tool that improves browsing performance while giving users greater privacy, security, and control over their web experience.



Requests are blocked before they finish loading.

6. Architecture pattern and justification

Ad-Vocates uses layered, pipeline, and client-server architecture.

The main application utilizes the layered pattern. The Dashboard serves as the presentation layer, with the application components handling functions, and Local Storage handling saved data. This matches the layered pattern of separating the presentation, logic, and data aspects of an application.

The pipeline pattern is utilized for the filter updating functionality. When filters are downloaded, they must move through the Filter Updater, Filter Validator, and Settings components before they will be used for filtering.

The client-server pattern is utilized for the download functionality of the Filter Updater. Ad-Vocates acts as a client, requesting a filter list from an approved source, and downloads the data from an external server.

These patterns were chosen because of the following criteria:

Fit to the problem: This architecture is fit to the problem, since these patterns match the way that the system works. The layered pattern separates the user interface, application functions and filtering logic, and storage which fits the desired design of the ad-blocker application. The pipeline pattern fits the process created to update filters, following a multi-step download and validation pipeline prior to allowing filters to be used. The client-server pattern is also a fitting pattern for governing the downloading of the filters from external approved sources.

Team skills: For a small, two-person team, these patterns fit well since they are not too complex to be handled by a small team size. As well, they create clear division in the components and architecture of the application, allowing for clean division of work and priority among the two members.

Performance and timing: Local filtering is used so requests can be processed quickly, and statistic reporting is separate from the filtering process to ensure quick processing of requests.

Scalability: Since Ad-Vocates is ran locally for each user, adding more users does not cause a server-strain issue, making scalability more practical.

Hardware constraints: Ad-Vocates will run on a user's machine and should consume little resources of the user's machine.

One pattern which was rejected was the microservices pattern. The reason why this pattern was rejected was because Ad-Vocates is not a large enough project to warrant the use of a microservice architecture. This architecture would be too complex and add little value to this type of project, and is more suitable for large systems.

7. Decision log

Decision	Alternatives considered	Reasoning for choice
Filtering is performed locally.	Sending requests to a remote filtering server.	Local filtering avoids network trip and keeps browsing information on the device.
The Filter Updater and Filter Validator will be separate.	One component downloads and checks filters.	A single component handling downloading and validation would be too much responsibility for a single component.
Statistics will collect blocked totals rather than browsing history data.	Save individual browsing requests	Gathering totals of blocked counts provided the information the Dashboard needs to present statistics, while reducing the information collected.
Use a website allowlist within settings	Pop-ups to allow individual websites, disable protection fully	Allowing the user to create their own allowlist can let users access websites broken by filtering in a flexible manner. Pop-ups lack persistence, and fully disabled protection increases risk for users and creates a weaker ad-blocker.